

FICHE FONDAMENTAUX — LANGAGE GO

Syntaxe, types, contrôle, fonctions, structures, concurrence

1. Présentation générale

- Créé par Google (2009) : Robert Griesemer, Rob Pike, Ken Thompson.
- Langage **compilé**, **typé statiquement**, avec **ramasse-miettes** (garbage collector).
- Syntaxe volontairement simple et minimaliste (peu de mots-clés, pas de classes, pas d'héritage).
- Concurrency intégrée nativement au langage : **goroutines** et **channels**.
- Compilation rapide vers un binaire natif autonome (pas de VM/interpréteur à l'exécution).
- Usages typiques : services réseau/serveurs, outils CLI, cloud/infra (Docker, Kubernetes sont écrits en Go).

Structure minimale d'un programme

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, world!")
}
```

- Tout fichier commence par **package** (« main » = programme exécutable).
- **import** déclare les paquets utilisés ; import inutilisé = erreur de compilation.
- Le point d'entrée est toujours la fonction **func main()** du package main.

2. Variables et types de base

```
var x int = 10          // déclaration explicite
var y = 10              // type inféré (int)
z := 10                // déclaration courte (uniquement dans une fonction)
var a, b int = 1, 2     // déclaration multiple
var c string            // valeur zéro : ""
```

Types de base

Catégorie	Types	Valeur zéro
Entiers	int, int8/16/32/64, uint, uint8(byte)...	0
Flottants	float32, float64	0.0
Booléen	bool	false
Chaîne	string	"" (vide)
Complexe	complex64, complex128	0+0i
Caractère	rune (= int32, un point de code Unicode)	0

- Chaque variable déclarée doit être utilisée, sinon erreur de compilation (sauf « _ » identificateur vide).
- Pas de conversion implicite entre types numériques : il faut convertir explicitement, ex. `float64(x)`.
- **Constantes** : `const Pi = 3.14` — valeur connue à la compilation, non modifiable.
- **iota** génère des constantes énumérées incrémentales :

```
const (
    Lundi = iota // 0
    Mardi  // 1
    Mercredi // 2
)
```

3. Structures de contrôle

if / else (pas de parenthèses, accolades obligatoires)

```

if x > 10 {
    fmt.Println("grand")
} else if x == 10 {
    fmt.Println("égal")
} else {
    fmt.Println("petit")
}

// forme avec initialisation locale
if n := calcul(); n > 0 {
    fmt.Println(n)
}

```

for — la seule boucle du langage (remplace while/do-while)

```

for i := 0; i < 10; i++ {    // forme classique
    fmt.Println(i)
}

for cond {                  // équivalent d'un while
    ...
}

for {                       // boucle infinie
    ...
    break
}

for i, v := range slice {   // parcours (index, valeur)
    fmt.Println(i, v)
}

```

switch (pas de fallthrough implicite, contrairement à C/Java)

```

switch jour {
case "lundi", "mardi":
    fmt.Println("début de semaine")
case "vendredi":
    fmt.Println("fin de semaine")
default:
    fmt.Println("autre")
}

```

4. Fonctions

```

func addition(a int, b int) int {
    return a + b
}

// paramètres de même type regroupés
func addition2(a, b int) int { return a + b }

// retours multiples (très utilisé pour la gestion d'erreurs)
func diviser(a, b int) (int, error) {
    if b == 0 {
        return 0, fmt.Errorf("division par zéro")
    }
    return a / b, nil
}

// fonction variadique
func somme(nombres ...int) int {
    total := 0
    for _, n := range nombres { total += n }
    return total
}

// fonction anonyme / closure
carre := func(x int) int { return x * x }

```

- Une fonction peut retourner **plusieurs valeurs** : très utilisé pour renvoyer (résultat, erreur).

- Les **closures** capturent les variables de leur environnement englobant.
- Pas de surcharge de fonctions (overloading) en Go.

5. Types composés : tableaux, slices, maps, structs

Tableaux (taille fixe) et slices (dynamiques, les plus utilisés)

```
var arr [5]int           // tableau de taille fixe = 5
s := []int{1, 2, 3}     // slice littéral
s = append(s, 4)        // ajouter un élément
sous := s[1:3]          // sous-slice [indice début:fin[
longueur := len(s)
capacite := cap(s)
s2 := make([]int, 3)     // slice de 3 zéros
```

Maps (tables associatives)

```
m := map[string]int{"a": 1, "b": 2}
m["c"] = 3
valeur, existe := m["a"]    // existe = true/false
delete(m, "b")
```

Structs (pas de classes en Go : structs + méthodes)

```
type Personne struct {
    Nom string
    Age int
}

p := Personne{Nom: "Ainhua", Age: 21}
fmt.Println(p.Nom)
```

6. Pointeurs

```
x := 10
p := &x           // p contient l'adresse de x (*int)
*p = 20           // déréférencement : modifie x -> x vaut 20
fmt.Println(x) // 20
```

- & = obtenir l'adresse ; * = déréférencer.
- Pas d'arithmétique des pointeurs (contrairement au C) : plus sûr.
- Utile pour modifier une valeur passée à une fonction (Go passe les arguments **par valeur** par défaut).

7. Méthodes et interfaces

Méthodes (fonctions avec un « receiver »)

```
func (p Personne) SeSaluer() string {
    return "Bonjour, je suis " + p.Nom
}

// receiver pointeur : permet de modifier la struct d'origine
func (p *Personne) Vieillir() {
    p.Age++
}
```

Interfaces (typage structurel implicite, pas de mot-clé « implements »)

```
type Animal interface {
    Crier() string
}

type Chien struct{}
func (c Chien) Crier() string { return "Wouf" }

// Chien satisfait automatiquement l'interface Animal
var a Animal = Chien{}
```

- Une interface est satisfaite **implicitement** dès qu'un type possède toutes ses méthodes.
- L'interface vide `interface{}` (ou any depuis Go 1.18) accepte n'importe quel type.
- Pas d'héritage : la **composition** (embedding de struct) remplace l'héritage classique.

8. Gestion des erreurs

- Go n'a pas d'exceptions classiques : les erreurs sont des **valeurs de retour** de type `error`.
- Convention : l'erreur est toujours le **dernier** résultat retourné, et on la teste juste après l'appel.

```
resultat, err := diviser(10, 0)
if err != nil {
    fmt.Println("erreur :", err)
    return
}
fmt.Println(resultat)
```

- **panic / recover** existent pour les erreurs vraiment exceptionnelles (rares, pas pour le contrôle de flux normal).
- **defer** : exécute une instruction juste avant que la fonction ne retourne (souvent pour fermer un fichier/connexion).

```
f, err := os.Open("fichier.txt")
if err != nil { return err }
defer f.Close() // exécuté à la sortie de la fonction, quoi qu'il arrive
```

9. Concurrency : goroutines et channels

- **Goroutine** : fonction exécutée de façon concurrente, très légère (mot-clé `go`).
- **Channel** : tuyau typé pour communiquer entre goroutines de façon sûre (au lieu de mémoire partagée + verrous).

```
func direBonjour(ch chan string) {
    ch <- "Bonjour depuis la goroutine" // envoi dans le channel
}

func main() {
    ch := make(chan string)
    go direBonjour(ch) // lance la goroutine
    message := <-ch // réception (bloquante) depuis le channel
    fmt.Println(message)
}
```

- Devise Go : « Ne communiquez pas en partageant de la mémoire ; partagez la mémoire en communiquant. »
- `sync.WaitGroup` permet d'attendre la fin de plusieurs goroutines.
- `select` permet d'attendre sur plusieurs channels à la fois (comme un switch pour channels).

10. Packages, modules et outils

- Un **module** Go se déclare avec `go mod init nom_module` (fichier `go.mod` généré).
- `go run fichier.go` : compile et exécute directement.
- `go build` : produit un binaire exécutable.
- `go get` : télécharge une dépendance externe.
- `go fmt` : reformate automatiquement le code selon le style officiel Go (indentation, espaces...).
- Convention de visibilité : un identificateur commençant par une **majuscule** est exporté (public) hors du package ; minuscule = privé au package.

À retenir — pièges fréquents

- Une variable ou un import déclaré mais non utilisé provoque une **erreur de compilation** (pas juste un warning).
- `:=` ne fonctionne que pour une **nouvelle** variable et uniquement à l'intérieur d'une fonction.
- Les slices partagent parfois le même tableau sous-jacent : modifier une sous-slice peut affecter l'original.
- Pas de `while`, pas de `do-while`, pas de ternaire `?:` : tout passe par `for` et `if`.
- Pas de classes ni d'héritage : struct + méthodes + interfaces + composition.
- Toujours vérifier `err != nil` juste après un appel qui retourne une erreur — ne jamais l'ignorer silencieusement.